

## Lab 3: Architecture Selection & Justification

Hostel Maintenance & Issue Ticketing System (Problem Statement #07)

Sanjeev Murthy – PES1UG24CS612

### Architecture Selection

---

We chose **Layered Architecture** for the **Hostel Maintenance & Issue Ticketing System**.

### Architectural Choice

---

We selected a three-layer style: a **Presentation Layer** (Ticket UI Component for the Hostel Resident, Maintenance Warden and Maintenance Staff), a **Business Layer** (Auth & Access Control, Ticket Manager, SLA & Escalation Monitor, Notification Service) and a **Data Layer** (Hostel Database, Attachment Store). The SMS/Email Gateway is an external system. Each layer talks only to the layer below it, through the provided/required interfaces shown in the component diagram.

### Two Reasons for the Choice

---

1. **The workflow is one transactional pipeline.** Every ticket follows the same path (FR-001 to FR-005): the resident logs it with room number, photo and severity, it is routed to the Warden queue, the warden assigns maintenance staff, staff update the status, and the resident gives closure sign-off, with the SLA tracked throughout. Layered architecture keeps all these rules in one place (Ticket Manager) with a single relational database, so a ticket is never half-created or in two states at once. A microservices design would need distributed transactions for the same guarantee.
2. **Right-sized for a hostel-scale system.** The load is a few hundred to a few thousand residents and a small warden team, built by a small student team. One deployable unit is easy to build, test and demo, while microservices would add API gateways, service discovery and per-service deployments that this scale does not need. Plain client-server would work but gives no internal structure, so Notification or SLA logic would get tangled with ticket logic. Because every layer depends only on interfaces (e.g. INotification, ITicketRepository), the SMS provider or the database can be swapped without touching other components.

### Security Advantage

---

The strict layer rule gives one security choke point: the UI can never reach the database directly. Every request first passes through the Auth & Access Control Component (IAuthentication, IAccessControl) in the Business Layer, which enforces role-based permissions as required by NFR-002 (a resident can only log, view and sign off their own tickets, only the warden can assign staff, and only the assigned maintenance staff can update ticket status) and validates inputs such as the mandatory room number before any data access. Because only the Business Layer holds database credentials, a compromised or tampered client cannot read or modify ticket data on its own.

### Performance Benefit

---

Components inside the Business Layer call each other in-process, so there is no network hop between Ticket Manager, SLA Monitor and Notification Service, which a microservices design would pay on every request. This keeps ticket creation, assignment and status updates fast, and the SLA tracking (FR-004) and 24-hour escalation check (NFR-001) run as a scheduled background job that does not slow down resident requests.

### Trade-off Acknowledged

---

Layered architecture scales as one unit rather than per component. This is acceptable at hostel scale, and if load grows, the Notification Service or SLA Monitor can be split out later since they already sit behind clean interfaces.